Edge Diffusion Speculative Decoding: Goodput Model & Gradient Scheduling

Quan La

December 23, 2025

Goal

- We follow GoodSpeed's abstraction: **time slots**, **draft length allocation** $S_i(t)$, **acceptance-rate estimation** $\hat{\alpha}_i(t)$, and **utility-based gradient scheduling**.
- Our twist: edge uses **diffusion drafter** and generates **best-of-K** candidates, but cloud **verifies only one selected draft**.

Key Alignment

GoodSpeed variable $S_i(t)$ (draft length) $\longleftrightarrow$ our $\gamma_i(t)$ (length of the selected draft sequence).

System Model

Entities

- N draft servers (edge clients) indexed by $i \in \{1, \ldots, N\}$.
- One verification server (cloud) hosting target model P.
- Each edge client hosts diffusion drafter Q_i.

Slot $t = 0, 1, 2, \ldots$

1. Cloud chooses draft length allocation $S_i(t) \in \mathbb{Z}_+$ for each client (and optionally best-of-K parameter $K_i(t)$).
2. Client i generates drafts on edge, selects one candidate, sends selected draft tokens to cloud.
3. Cloud verifies tokens and commits accepted prefix (speculative decoding).

Edge Drafting: Best-of-K (Local to Client)

Edge candidate generation

Client i with prompt/context $x_i(t)$ generates $K_i(t)$ candidates:

$$\boldsymbol{y}_i^{(k)}(t) = \left(y_{i,1}^{(k)}(t), \ldots, y_{i,S_i(t)}^{(k)}(t)\right), \quad \boldsymbol{y}_i^{(k)}(t) \sim Q_i(\cdot \mid x_i(t)), \quad k = 1..K_i(t).$$

Best-of-K selection

$$k_i^*(t) = \underset{k \in [1..K_i(t)]}{\arg\max} \ \texttt{TokensDraft}\left(x_i(t), \boldsymbol{y}_i^{(k)}(t)\right), \qquad \tilde{\boldsymbol{y}}_i(t) = \boldsymbol{y}_i^{(k_i^*)}(t).$$

Only $\tilde{\boldsymbol{y}}_i(t)$ is verified by the cloud.

Cloud Verification & Realized Goodput

Acceptance indicators

For the selected draft $\tilde{\boldsymbol{y}}_i(t) = (\tilde{y}_{i,1}(t), \ldots, \tilde{y}_{i,S_i(t)}(t))$:

$$A_{i,j}(t) = \mathbf{1}\{\tilde{y}_{i,j}(t) \text{ is accepted by verifier } P\}, \quad j = 1..S_i(t).$$

Accepted prefix length:

$$\ell_i(t) = \max\{\ell \leq S_i(t) : A_{i,1}(t) = \cdots = A_{i,\ell}(t) = 1\}.$$

Goodput definition

We count accepted draft tokens plus one correction token:

$$x_i(t) \triangleq 1 + \ell_i(t).$$

(If you do not count correction, set $x_i(t) = \ell_i(t)$.)

Goodput Model $\mu(S, \alpha)$

Effective acceptance rate

Best-of-K alters acceptance statistics; we absorb it into an **effective** per-token acceptance rate:

$$\alpha_i(t) \equiv \alpha_i^{\text{eff}}(x_i(t), K_i(t)) \in (0, 1).$$

Expected goodput for draft length S

Under i.i.d. acceptance with rate α:

$$\mu(S, \alpha) \triangleq \mathbb{E}[x \mid S, \alpha] = \sum_{j=0}^{S} \alpha^j = \frac{1 - \alpha^{S+1}}{1 - \alpha}.$$

Thus, predicted goodput for client i in slot t:

$$\hat{x}_i(t) \approx \mu\big(S_i(t), \hat{\alpha}_i(t)\big).$$

Online Estimation of Acceptance Rate (Smoothing)

Empirical acceptance estimate from realized accepts

We estimate acceptance on the selected draft:

$$\tilde{\alpha}_i(t) = \frac{1}{S_i(t)} \sum_{j=1}^{S_i(t)} A_{i,j}(t).$$

Exponential smoothing

$$\hat{\alpha}_i(t) = (1-\eta)\hat{\alpha}_i(t-1) + \eta\,\tilde{\alpha}_i(t), \qquad \eta \in (0,1).$$

- Interpretation: $\hat{\alpha}_i(t)$ is the current effective accept-rate after best-of-K selection.

Long-term Performance Variable $X_i(t)$

Smoothed goodput

Maintain a moving average of realized goodput:

$$X_i(t) = (1-\beta)X_i(t-1) + \beta\, x_i(t), \qquad \beta \in (0,1).$$

- $X_i(t)$ is the state used by the scheduler (for fairness via log utility).

Resource Constraint

Verification budget

Cloud verification capacity per slot:

$$\sum_{i=1}^{N} S_i(t) \leq C, \qquad S_i(t) \in \mathbb{Z}_+.$$

Define the feasible set:

$$\mathcal{K} \triangleq \left\{ \boldsymbol{S} \in \mathbb{Z}_+^N : \sum_{i=1}^{N} S_i \leq C \right\}.$$

Best-of-K cloud cost

If cloud also scores K candidates, add a second budget:

$$\sum_{i=1}^{N} K_i(t)\, S_i(t) \leq C_{\text{score}}.$$

Utility Maximization

Utility

Let $U_i(\cdot)$ be concave, increasing ($U_i(x) = \log x$ for proportional fairness). Goal: maximize long-term sum utility of goodput.

Gradient weights

GoodSpeed uses instantaneous gradients:

$$w_i(t) \triangleq \nabla U_i\big(X_i(t)\big).$$

Gradient Scheduling

At each slot, cloud selects next allocation $\boldsymbol{S}(t+1) \in \mathcal{K}$ by:

$$\boldsymbol{S}(t+1), K(t+1) \in \arg\max_{\boldsymbol{S} \in \mathcal{K}} \sum_{i=1}^{N} w_i(t)\, \mu\big(S_i, \hat{\alpha}_i(t)\big).$$

- Inputs: smoothed states $X_i(t)$ and acceptance estimates $\hat{\alpha}_i(t)$.
- Output: draft lengths $S_i(t+1), K_i(t+1)$ for each client.

Algorithm Summary

1. **Observe** states $(X_i(t), \hat{\alpha}_i(t))$ for all clients.
2. **Allocate** draft lengths $\boldsymbol{S}(t+1), K(t+1)$ by gradient scheduling:

$$\max_{\boldsymbol{S} \in \mathcal{K}} \sum_i w_i(t)\, \mu(S_i, \hat{\alpha}_i(t)).$$

3. **Edge drafting**: each client generates $K_i(t+1)$ candidates and selects $\tilde{\boldsymbol{y}}_i(t+1)$.
4. **Verify**: cloud verifies selected drafts.
5. **Update**: $(X_i, \hat{\alpha}_i)$ via exponential smoothing.